

Regular Emails for Regular People

Moham Farah Ballfucky^{*1} and Violet Szabó^{†1}

¹The Regularists

March 18, 2026

Abstract

Programmers are often tasked with the problem of determining whether an email address is valid or not. Regular people have long used regular expressions to validate email addresses, and this method has generally prospered. However, a faction of Irregularists has emerged, which has been arguing that this method is improper: they argue that the language of email addresses is, in fact, formally context-free due to issues such as parenthesis matching, and thus regular expressions should not be used to validate them. In this paper, we show that this view is flawed: not only is the language of email addresses regular in the formal sense, but regular expressions are the most appropriate way to match any emails compliant with the relevant standards.

1 Irregular Delusions

It has long been believed that email addresses can only be defined by a context-free grammar. This is due to propaganda by Bert, the President of the Irregularists, who has been quoted as spreading lies such as

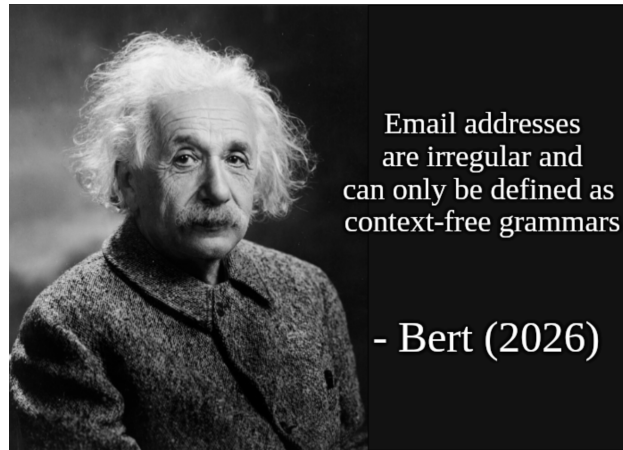


Figure 1: Bert spreading lies and deceit

Bert claimed that since CFGs could contain comments within parentheses, e.g. `((like))this@valid.email`, they then could not be defined as regular languages, as parenthesis matching is notoriously irregular.

^{*}(This is my escape)mbalfakeih(I'm running through this world)@(and I'm not looking back 'Cause I know I can go)protonmail.com(Where no one's ever gone and I'm not looking back But how will I know when I get there? And how will I know when to leave?We've all gotta start from somewhere And it's right there for me)

[†](According to all known laws of aviation,)viszabo(there is no)@(way a bee should be able to fly. Its wings are too small to get its fat little body off the ground. The bee, of course, flies anyway because bees don't care what humans think is impossible. Yellow, black)protonmail.com(Yellow, black. Yellow, black.)

All code can be found at: <https://github.com/mbalfakeih/RegMail>

However, the benevolent, gracious, and regular writers of RFC 5322 and RFC 5321 did not intend for this to be the case! And have long fought against the Irregularists' Propaganda! They *specifically* wrote in RFC 5321 Section 4.5.3.1.1. and RFC 5321 Section 4.5.3.1.2. that the local part of an email (before the @) and the domain of an email (after the @) must be 64 and 255 octets respectively.

The explicit finitude of email addresses can only mean one thing, they are regular. With this knowledge, we, The Regularists, embark on a monumental and honorable quest to create a truly, formally regular expression for email addresses.

2 The Irregularity Pandemic

As much as it pains us to discuss the current state of irregularity, it is crucial for us to shed light on this matter to ensure all regular people understand the gravity of our results.

The CFG for email addresses is defined under RFC 5322 Section 3.4. We define a regex for the `addr-spec` rule, as it is what regular people think of as an email address.

Moreover, RFC 5322 defines a set of obsolete syntax that email clients must never generate, but must accept. Some may argue that we should move forward and disregard this obsolete syntax to aid in the quest for regularization. However, that would be discriminatory to my regular grandma, who likes putting little comments in her email address, and that would make her sad. For this reason, we will ensure our regex matches all possible RFC 5322-compliant email addresses.

The current state of email clients is, to put it bluntly, horrendous: different email clients support different features of email addresses, and are terribly inconsistent. For example, based on our rigorous testing of three email clients, Proton, Outlook, and Gmail, only Proton was able to send emails with comments, and all email clients could send emails with quoted strings, e.g. "like this!"@hi.hello. However, Outlook cannot receive emails with quoted strings, indicating that it may occasionally forget it supports this feature.

The Irregularists may point out that it is fairly simple to construct a parser for a finite CFG, and therefore it is unnecessary to create a regular expression. However, most email clients already use incomplete regular expressions to check email address validity, showing a clear preference for regularity. Moreover, the Irregularists are using an unnecessarily powerful model of computation when they could be using the simple and pure power of regular languages to perform their task. To unnecessarily avoid the use of regularity is confusing and upsetting to all.

As we have shown, the current landscape of email addresses is wildly irregular and inconsistent. This motivates our quest to find a singular, purely regular expression for all email clients to use, so that the world of email addresses can be at peace for many years to come.

3 The Regularization

The crux of our quest involves transforming the CFG defined in RFC 5322 into a regular expression. This is done through a regular 3-step process:

1. Convert the CFG into Chomsky Normal Form
2. Convert the CFG in CNF into a finite CFG in CNF
3. Convert the finite CFG in CNF into a regular expression

We perform Step 1 through a process called "manual labor" and "not feeling like looking for pre-existing CFG libraries in Haskell". This process was performed perfectly immediately.

Step 2 is performed by setting a budget on production rules of the CFG; in our case, a budget of 64 is placed on the local part and 255 on the domain. For the sake of example and demonstration, consider this simple grammar in CNF:

$$A \rightarrow BC$$

$$B \rightarrow x$$

$$C \rightarrow y$$

Say the budget of production rule A is 2, then this means that the production rules B C have a maximum of 2 characters together, this can be split as B having a budget of 0 characters and C having a budget of 2 characters, B having a budget of 1 character and C having a budget of 1 character, etc. We denote this as a superscript:

$$A^2 \rightarrow B^0 C^2$$

$$A^2 \rightarrow B^1 C^1$$

$$A^2 \rightarrow B^2 C^0$$

However, not all of these rules are possible; for example, we can eliminate C^0 , since C always results in a single terminal. This also eliminates the bottom rule entirely. Applying this process to the entire CFG, we get:

$$A^2 \rightarrow B^1 C^1$$

$$B^1 \rightarrow x$$

$$C^1 \rightarrow y$$

Of course, this can get significantly more complicated. But we prefer to show regular examples.

Step 3 is very simple, since the operations used in our CFG are concatenation and union, which both exist in a regular expression. We are able to regularly perform this transformation.

4 Results, or the power of regularity

The above procedure straightforwardly allows us to compute a regular expression to match any email address. Unfortunately, like the greats before us [1], we are faced with a lack of space to fit our results. However, as our procedure can simply be performed with a lower limit on email length, we give the reader a glimpse of the beauty of our solution.

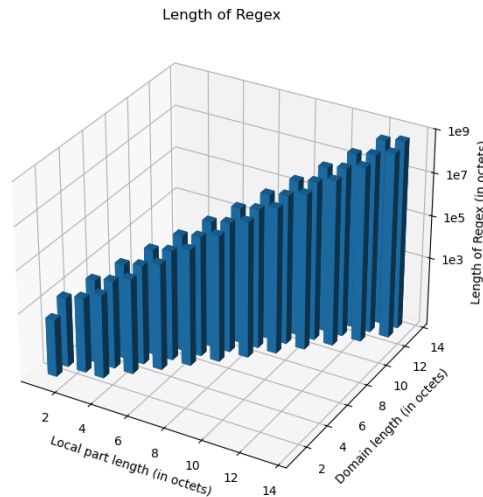


Figure 2: Generated regex size (logarithmic) per maximum length of the local part and domain. For improved readability, the graph is three-dimensional.

Figure 2 shows the size of the generated regular expression for a local part and domain of a given maximum length. All email addresses with a local part and domain of at most 5 bytes each can be matched by a 129 kB

regular expression. For email addresses with at most 13 bytes in either component, the expression measures at 557 MB. Based on an exponential regression, the full regular expression with a 64-byte local part and 255-byte domain would be around 1.55×10^{80} bytes long.

Irregularists may argue that this regular expression is several orders of magnitude larger than the total disk space available on planet Earth. We instead draw attention to the fact that our regular expression is, in fact, staggeringly efficient: even assuming that users only use simple email addresses containing only lowercase ASCII letters, there are still more than $26^{319} \approx 2.4 \times 10^{451}$ valid email addresses. This means that the compression ratio of our regular expression is $10^{371} : 1$. As an analogy, the diameter of the observable universe is estimated to be around 8.8×10^{26} meters: were we to apply our compression scheme to it, it would become many orders of magnitude smaller than the Planck length, instantaneously eradicating all life, and thus all Irregularists.

These results highlight that while our work is limited by the current state of computer storage technology, the power of regularity is not to be underestimated. In today's world, it has become customary to appeal to 'nuance' and to think about 'details'; we instead believe that life should be simple, and that when you have a hammer, everything is a nail if you just try hard enough.

5 The Path to True Regularity

We envision a world in which all email addresses are parsed via regular expressions. It is clear that, at our current level of technological development, this is not yet feasible. However, we argue that this is due to incorrect priorities rather than any core technological limitation. Irregularists have long argued that overcomplicated tools and abstract models are necessary to conserve resources: as this paper has demonstrated, regularity is more than capable of handling emails. Therefore, we argue that the world would be better off if we spent our financial resources in pursuit of creating regular solutions to regular problems. Emails present a good first step in this direction.

Current email providers either use overcomplicated, irregular parsers or undercomplicated, incomplete regular expressions to validate email addresses. We are currently seeking grants to create an email service that uses the correct approach. Aside from being an objectively superior service, this would allow users to signal their commitment to regularity, strengthening the ties within the regular community.

References

- [1] Pierre de Fermat. *Diophanti Alexandrini Arithmeticonum Libri Sex*. Ed. by Samuel de Fermat. Toulouse: Apud Bernardum Bosc, 1670.